

Let $f: \Sigma_0^* \rightarrow \Sigma_0^*$. We say a TM M computes f if
for any $w \in \Sigma_0^*$, $M(w) = f(w)$ (M halts with $\triangleright \sqcup f(w)$)

computable
recursive

numerical function $f: N^k \rightarrow N$

We say a TM M computes a numerical function $f: N^k \rightarrow N$ if
for any $n_1, \dots, n_k \in N$,

$$M(\text{bin}(n_1); \text{bin}(n_2); \dots; \text{bin}(n_k)) = \text{bin}(f(n_1, \dots, n_k))$$

Basic functions

(1) for any $k \geq 0$, the k -ary zero function is

$$\text{zero}(n_1, \dots, n_k) = 0$$

for any $n_1, \dots, n_k \in N$

(2) for $k \geq j > 0$, the j th $\overset{\text{identity}}{\vee}$ k -ary function is

$$\text{id}_{kj}(n_1, \dots, n_k) = n_j$$

(3) the successor function

$$\text{succ}(n) = n+1$$

for any $n \in N$

Two operations

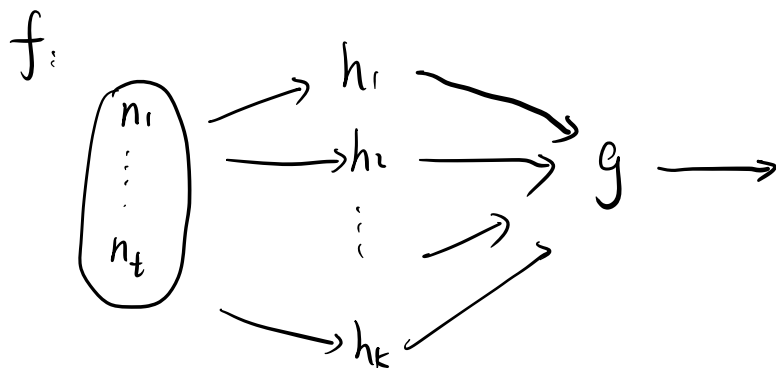
(1) composition

Given $g: N^k \rightarrow N$ for some $k \geq 0$, and

$h_1, \dots, h_k: N^t \rightarrow N$ for some $t \geq 0$.

the composition of g with $h_1, \dots, h_k$ is the t -ary function

$$f(n_1, \dots, n_t) = g(h_1(n_1, \dots, n_t), \dots, h_k(n_1, \dots, n_t))$$



$$\text{succ}(\text{succ}(n)) = n+2$$

(2) recursive definition

Given $g: N^k \rightarrow N$ and

$h: N^{k+1} \rightarrow N$ for some $k \geq 0$

the function defined recursive by g and h is the $(k+1)$ -ary function

base case $\rightarrow f(n_1, \dots, n_k, 0) = g(n_1, \dots, n_k)$

recurrence $\rightarrow f(n_1, \dots, n_k, m+1) = h(n_1, \dots, n_k, m, f(n_1, \dots, n_k, m))$

$$\text{plus}(m, n) = m+n$$

$$\text{plus}(m, 0) = \text{id}_{2,1}(m, 0) = m$$

$$\text{plus}(m, n+1) = \text{plus}(m, n) + 1 = \text{succ}(\text{plus}(m, n))$$

$$h(m, n, \text{plus}(m, n)) = \text{succ}(\text{id}_{3,3})(m, n, \text{plus}(m, n))$$

Let $f: N^k \rightarrow N$

Let $g: N^{t+k} \rightarrow N$ such that

$$g(\underline{m_1, \dots, m_t}, n_1, \dots, n_k) = f(\underline{n_1, \dots, n_k})$$

for any $m_1, \dots, m_t, n_1, \dots, n_k \in N$

$$g(m_1, \dots, m_t, n_1, \dots, n_k)$$

$$= f \left(\begin{array}{l} \text{id}_{t+k, t+1}(m_1, \dots, m_t, n_1, \dots, n_k), \\ \text{id}_{t+k, t+1}(\\ \vdots \\ \text{id}_{t+k, t+k}(m_1, \dots, n_k) \end{array} \right), \begin{array}{l} // n_1 \\ // n_2 \\ \\ // n_k \end{array}$$

Definition

The primitive recursive functions are all the basic functions and all the functions obtained from the basic function by composition and recursive definition.

Corollary

Compositions of primitive recursive functions are primitive recursive. Functions defined recursively by primitive recursive functions are

Example of Primitive Recursive functions

(a) $\text{plus}_2(n) = n+2$

(b) $\text{plus}(m, n) = m+n$

(c) $\text{mult}(m, n) = m \cdot n$

(d) $\text{exp}(m, n) = m^n$

(e) constant function

$$f(n_1, \dots, n_k) = c \text{ for some } c \in \mathbb{N}$$

$$= \underbrace{\text{succ}(\dots \text{succ}(\text{zero}(n_1, \dots, n_k)) \dots)}_c$$

(f) sign function

$$\text{sgn}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n \geq 1 \end{cases}$$

$$\begin{aligned} \text{sgn}(0) &= 0 \\ \text{sgn}(n+1) &= 1 \end{aligned} \quad \begin{aligned} &g: \mathbb{N}^k \rightarrow \mathbb{N} \\ &h: \text{constant function} \end{aligned}$$

(g) predecessor function

$$\text{pred}(n) = \begin{cases} 0 & \text{if } n=0 \\ n-1 & \text{if } n \geq 1 \end{cases}$$

$$\begin{aligned} \text{pred}(0) &= 0 \\ \text{pred}(n+1) &= n = \overset{h}{\text{id}_{2,1}}(n, \text{pred}(n)) \end{aligned}$$

$$(h) \quad m \sim n = \max\{m-n, 0\} \quad (\text{non-negative subtraction})$$

$$\begin{aligned} m \sim 0 &= m = \text{id}(m) \\ m \sim (n+1) &= \text{pred}(m \sim n) \end{aligned}$$

primitive recursive

$$f(n_1, \dots, n_k) = \begin{cases} \dots & \text{if } \dots \\ \dots & \text{otherwise} \end{cases}$$

Definition

A primitive recursive function is a primitive recursive predicate if it only takes values of 0 and 1.

Example of Primitive Recursive Predicate.

$$(a) \quad \text{iszero}(n) = \begin{cases} 1 & \text{if } n=0 \\ 0 & \text{if } n > 0 \end{cases}$$

$$\text{iszero}(0) = 1$$

$$\text{iszero}(n+1) = 0$$

$$(b) \text{ ispositive}(n) = \begin{cases} 1 & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

↘ $\text{sgn}(n)$

$$(c) \text{ geg}(m, n) = \begin{cases} 1 & \text{if } m \geq n \\ 0 & \text{if } m < n \end{cases}$$

↘ $\text{iszero}(n \sim m)$

Theorem

1. the negation of a primitive recursive predicate is primitive recursive

$$\neg p(m) = 1 \sim p(m)$$

2. the disjunction and conjunction of two primitive recursive predicates are

$$p(m) \vee q(m) = 1 \sim \text{iszero}(p(m) + q(m))$$

$$p(m) \wedge q(m) = 1 \sim \text{iszero}(p(m) \cdot q(m))$$

3. Let $g, h : N^k \rightarrow N$ be primitive recursive functions

Let $p : N^k \rightarrow N$ be a primitive recursive predicate.

The following function is primitive recursive.

$$\begin{cases} g(n_1, \dots, n_k) & \text{if } p(n_1, \dots, n_k) \\ h(n_1, \dots, n_k) & \text{if } \neg p(n_1, \dots, n_k) \end{cases}$$

$$f(n_1, \dots, n_k) = \begin{cases} 1 & \text{if } (n_1, \dots, n_k) \text{ is prime} \\ h(n_1, \dots, n_k) & \text{otherwise} \end{cases}$$

$$f = p \cdot g + (1 \sim p) \cdot h$$

More example.

$$(a) \text{ eq}(m, n) = \begin{cases} 1 & \text{if } m=n \\ 0 & \text{if } m \neq n \end{cases}$$

$$\text{eq}(m, n) = \text{geg}(m, n) \wedge \text{geg}(n, m)$$

(b) $\text{rem}(m, n)$ = remainder of m divided by n

$$\text{rem}(0, n) = 0$$

$$\text{rem}(m+1, n) = \begin{cases} 0 & \text{if } \underline{\text{eq}(\text{rem}(m, n), \text{pred}(n))} \\ \text{rem}(m, n) + 1 & \text{otherwise} \end{cases}$$

Note: Red circles highlight '0' and 'rem(m, n) + 1'. A red arrow points from the 'otherwise' branch to the 'eq(pred(m, n), pred(n))' condition.

$$(c) \text{div}(m, n) = \lfloor m/n \rfloor \quad \text{div}(5, 2) = 2$$

$$\text{div}(0, n) = 0$$

$$\text{div}(m+1, n) = \begin{cases} \text{div}(m, n) + 1 & \text{if } \underline{m+1 \text{ is divisible by } n} \\ \text{div}(m, n) & \text{otherwise} \end{cases}$$

(d) $\text{digit}(m, n, p)$ = the m -th digit of the base p representation of n

$$n = \underbrace{a_k p^k + a_{k-1} p^{k-1} + \dots + a_{m-1} p^{m-1}}_{\text{}} + \dots + a_1 p + a_0$$

$$\text{digit}(m, n, p) = \text{div}(\text{rem}(n, p^m), p^{m-1})$$

(e) If $f(m, n)$ is primitive recursive, so is

$$\text{sum}_f(m, n) = \sum_{k=0}^n f(m, k)$$

A wrong proof:

$$\text{sum}_f(m, n) = f(m, 0) + f(m, 1) + \dots + f(m, n)$$

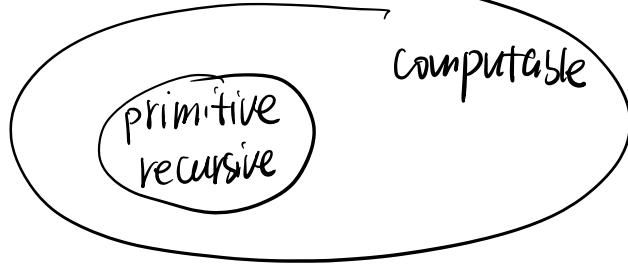
Correct proof:

$$\text{sum}_f(m, 0) = f(m, 0)$$

$$\text{sum}_f(m, n+1) = \text{sum}_f(m, n) + f(m, n+1)$$

primitive recursive = computable (recursive)?

X



Theorem

If a function is primitive recursive, it is computable.

Proof Idea:

Basic function

1. $zero(n)$
2. $id_{k,j}$ $\longrightarrow$ computable.
3. $succ(n)$

Two operation

1. composition
2. recursive definition $\longrightarrow$ preserve computability

Theorem

There is a function that is computable but not primitive recursive

claim

The set of all unary primitive recursive functions can be lexicographically enumerated by a TM.

Proof:

$M =$

1. enumerate all string in Σ^* in lexicographical order
2. every time a string is enumerated

output it if it is a valid encoding of a unary
primitive recursive function

Name all the unary primitive recursive function as
 $f_0, f_1, f_2, f_3, \dots$

define f^* such that $\forall n \in \mathbb{N}$
 $f^*(n) = f_n(n) + 1$

$$\forall n \in \mathbb{N}, f^*(n) = f_n(n) + 1 \neq f_n(n)$$

$$\Rightarrow f^* \neq f_n \text{ for } \forall n \in \mathbb{N}$$

$\Rightarrow f^*$ is not primitive recursive.

f^* is computable.

$\curvearrowright M$: on input n

1. enumerate all the unary primitive recursive function
until it outputs f_n

2. output $f_n(n) + 1$

$$\downarrow$$
$$f^*(n)$$

Observation

Let S be a subset of all the numerical functions.

If the following problem is decidable

given a numerical function f , $\rightarrow$

is $f \in S$?

Then $S \neq$ the set of computable numerical function.

Proof:

If some function in S is not computable, $---$ holds

If all the functions in S are computable, repeat the previous proof,

we can construct a function f^* , $f^* \notin S$.

computable,

↑